Ενότητα 06:
The Pipeline III

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy

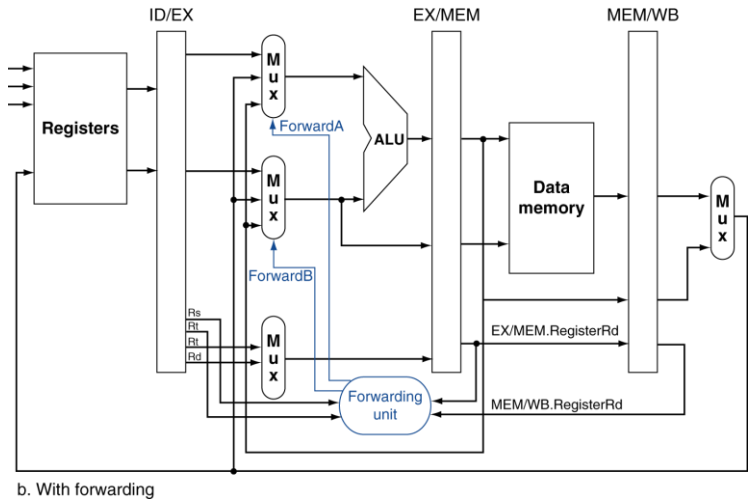
Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

1

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

Forwarding Paths



b. With forwarding

2

Double Data Hazard

- Υποθέστε το σενάριο:

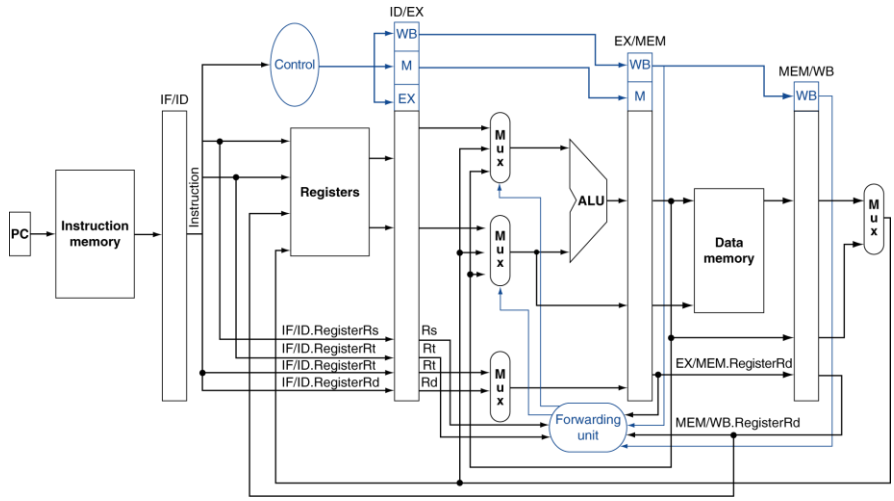
```
add $1,$1,$2
add $1,$1,$3
add $1,$1,$4
```

- Έχουμε 2 hazards συνεχόμενα
  - Άρα αν ενεργοποιηθεί το forwarding σε όλα τα στάδια από την 2η εντολή τότε η 3η εντολή πια τιμή θα πάρει?
  - Πρέπει να βεβαιωθούμε ότι κάθε φορά που κάνουμε forwarding στέλνουμε τη σωστή εκδοχή του καταχωρητή/αποτελέσματος
- Αλλάζουμε το MEM Hazard condition:
  - Κάνε forward μόνο εάν το EX hazard condition δεν είναι true

Revised Forwarding Condition

- MEM hazard
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs))
    ForwardA = 01
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRt))
    ForwardB = 01

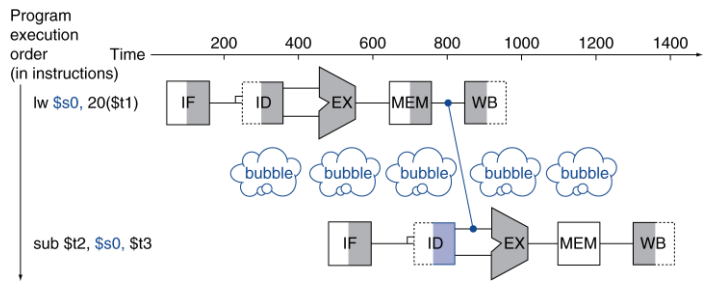
Datapath with Forwarding



5

Load-Use Data Hazard

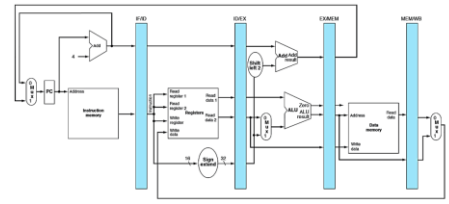
- Δεν μπορούμε πάντα να χρησιμοποιούμε το Forwarding
  - Εάν η τιμή ακόμα δεν έχει παραχθεί ή καθυστερήσει τότε το forwarding δεν θα δουλέψει
  - Αυτό συμβαίνει συνήθως με Load εντολές που πρέπει να φέρουν δεδομένα από τη μνήμη που μπορεί να πάρει περισσότερους από 1 κύκλο



6

Load-Use Hazard Detection

- Ελέγχουμε το είδος των εντολών στο στάδιο ID
- ALU operand registers στο στάδιο ID δίνονται από
  - IF/ID.RegisterRs
  - IF/ID.RegisterRt
- Load-use hazard όταν:
  - ID/EX.MemRead and
    ((ID/EX.RegisterRt = IF/ID.RegisterRs) or
    (ID/EX.RegisterRt = IF/ID.RegisterRt))

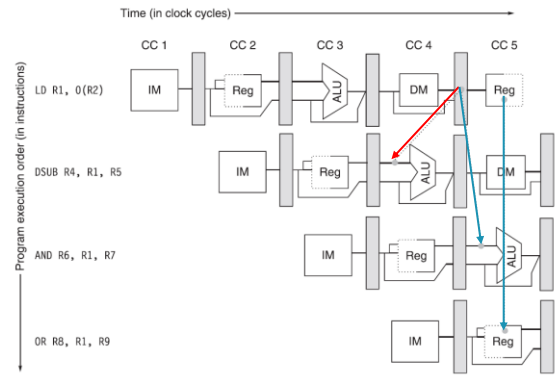


- Εάν βρούμε ένα Load-Use Hazard τότε καθυστερούμε το pipeline με bubbles (**no-ops**) δημιουργώντας stalls

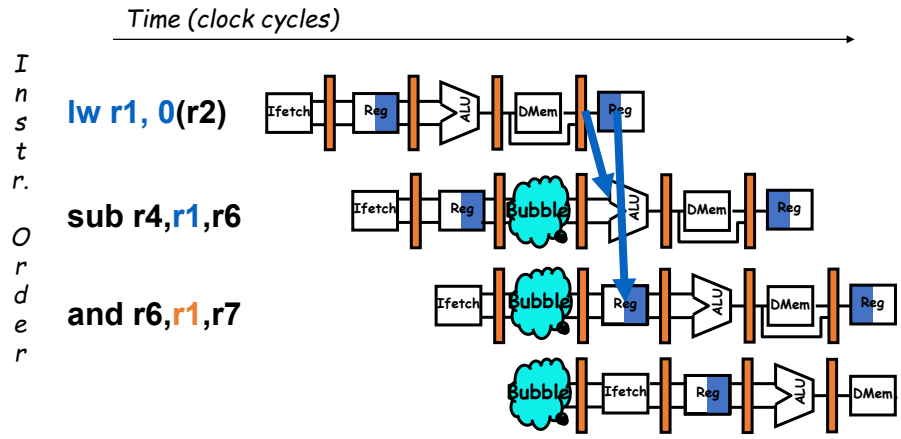
Data Hazards solutions: **Stalling**

- Προσωρινή παύση του pipeline για να περιμένει να γίνουν διαθέσιμα τα απαραίτητα δεδομένα

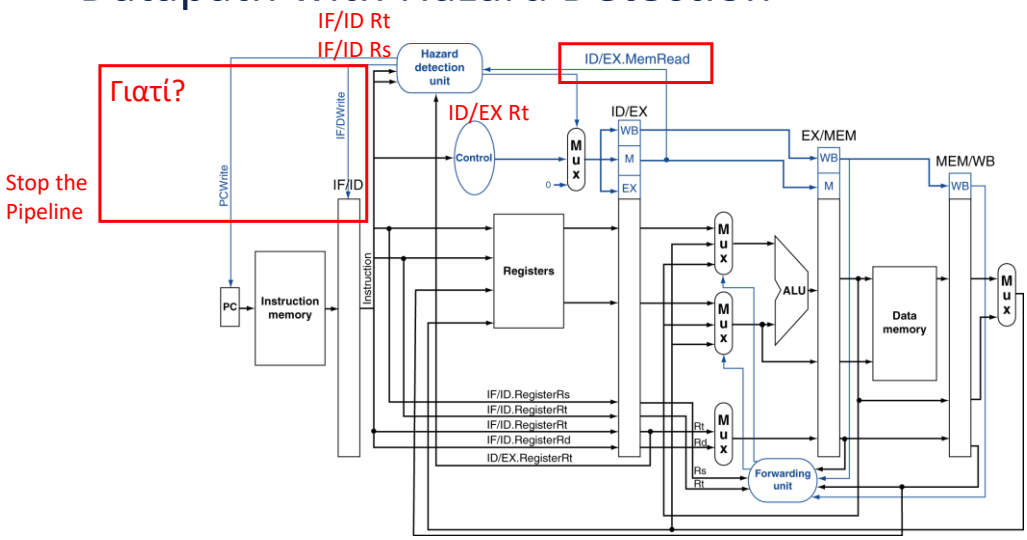
```
ld      x1,0(x2)
sub     x4,x1,x5
and     x6,x1,x7
or      x8,x1,x9
```



Παράδειγμα **Stalling**



Datapath with Hazard Detection

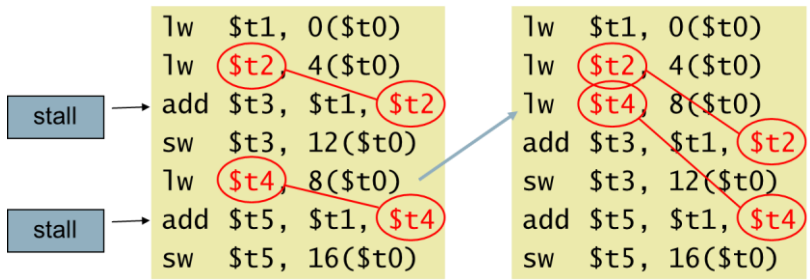


Επίδοση με Stalls

- Τα stalls μειώνουν σοβαρά την επίδοση!!!
  - Αλλά κάποιες φορές είναι αναγκαία για να έχουμε σωστά αποτελέσματα του προγράμματος
- Υπάρχουν και άλλες λύσεις όμως:
  - Code Reordering
    - Από τον Compiler
    - Αποφεύγει τα stalls (δεν είναι όμως 100% επιτυχής)
    - Χρειάζεται γνώση του pipeline και των σταδίων του

Code Reordering

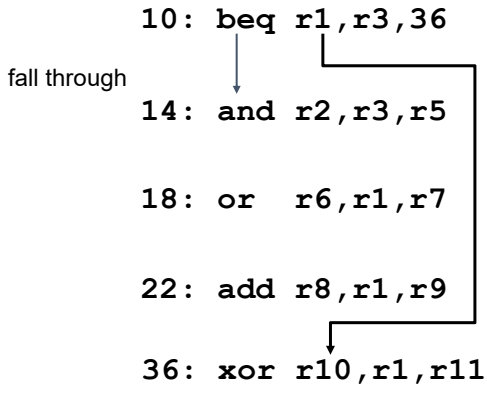
- Αλλάζουμε την σειρά των εντολών για να αποφύγουμε την χρήση του $t2 και $t4 στις αμέσως επόμενες εντολές
- Κώδικας C: `A = B + E; C = B + F;`



Control Hazards

- Προκύπτουν όταν το pipeline δεν μπορεί να καθορίσει την επόμενη εντολή προς ανάκτηση (IF) λόγω μιας εντολής branch ή jump
- Αυτές οι εντολές μπορούν να αλλάξουν το PC, ο οποίος διακόπτει τη ροή της διαδοχικής ανάκτησης εντολών (IF)
- Το αποτέλεσμα μιας εντολής branch μπορεί να μην είναι γνωστό μέχρι να εκτελεστεί η εντολή, αλλά η επόμενη εντολή έχει ήδη φορτωθεί από τη μνήμη

Παράδειγμα Control Hazard



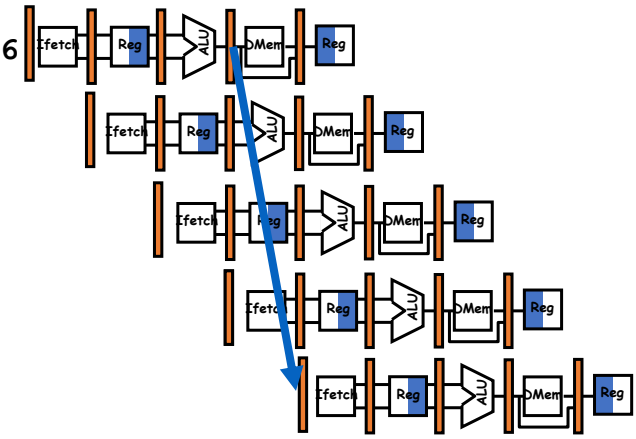

```
if (x == 0)
{
      x++;
      a = x + y;
      printf("%d", x);
}

x--;
printf("%d", x);

b = b + c;
b++;
```

Control Hazards

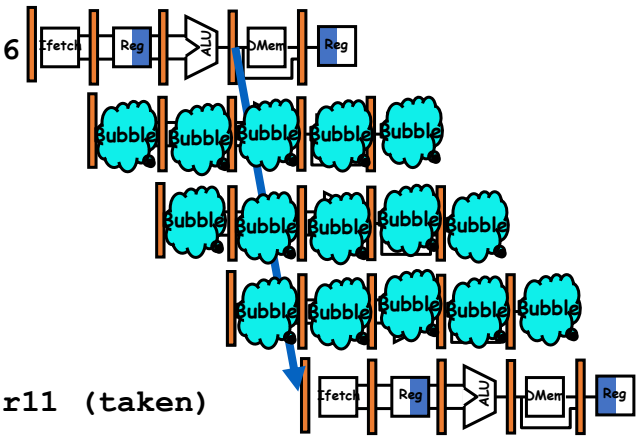
```
10: beq r1,r3,36
```



15

Control Hazard Solutions: **Stalling**

```
10: beq r1,r3,36
stall
stall
stall
36: xor r10,r1,r11 (taken)
     ή
14: and r2,r3,r5 (not taken)
```



16

Control Hazard Solutions: **Delay Branch**

- Περιλαμβάνει τον προγραμματισμό (scheduling) χρήσιμων εντολών που θα εκτελεστούν στα stall bubbles αμέσως μετά από μια εντολή branch
- Οι εντολές αυτές εκτελούνται ανεξάρτητα από το αποτέλεσμα διακλάδωσης
- Απαιτεί πολύπλοκες τεχνικές στον compiler που απαιτούν τον compiler να βλέπει πολύ μπροστά και συνήθως είναι αδύνατο να προβλέψουμε ποιες εντολές θα εκτελεστούν ανεξάρτητα από το branch

```
if (x == 0)
…
…
…
{
      x++;
      a = x + y;
      printf("%d", x);
}

x--;
printf("%d", x);
```

```
b = b + c;
b++;
```


Control Hazard Solutions: **Branch Prediction**

- Προβλέπουμε το αποτέλεσμα του Branch
- **Static Prediction:** Προβλέπουμε πάντα το ίδιο αποτέλεσμα (taken)
- **Dynamic Prediction:** Χρησιμοποιούμε ιστορικά δεδομένα για να κάνουμε εμπεριστατωμένες εικασίες (educated guesses)
  - Οι επεξεργαστές συνήθως διατηρούν έναν πίνακα Branch History Table (BHT) για την παρακολούθηση της πρόσφατης συμπεριφοράς των branch εντολών
  - **>90% accuracy!**

Ενότητα 06:
The Pipeline III

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy

Τεχνολογικό Πανεπιστήμιο Κύπρου
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

19